

Buyer Setup Guide

Welcome to your new E-Commerce Automotive Store! This comprehensive guide will walk you through setting up your store from scratch.

Table of Contents

1. [Prerequisites](#)
 2. [Quick Start](#)
 3. [Detailed Setup Instructions](#)
 4. [Environment Variables Explained](#)
 5. [Database Setup](#)
 6. [Testing Your Installation](#)
 7. [Common Setup Issues](#)
-

Prerequisites

Before you begin, ensure you have the following installed on your system:

Required Software

- **Node.js** (v18.0.0 or higher)
 - Download from: <https://nodejs.org/>
 - Verify installation: `node --version`
- **npm** (v9.0.0 or higher) - comes with Node.js
 - Verify installation: `npm --version`
- **PostgreSQL** (v14.0 or higher)
 - Download from: <https://www.postgresql.org/download/>
 - Verify installation: `psql --version`
- **Git** (for cloning the repository)
 - Download from: <https://git-scm.com/>
 - Verify installation: `git --version`

Optional but Recommended

- **pgAdmin** - GUI tool for PostgreSQL database management
 - **VS Code** - Recommended code editor
 - **Stripe Account** - For payment processing (see STRIPE_SETUP_GUIDE.md)
-

Quick Start

For experienced developers who want to get started quickly:

```
# 1. Extract/Clone the template
cd your-project-directory

# 2. Install dependencies
npm install

# 3. Copy environment variables
cp .env.local.example .env.local

# 4. Edit .env.local with your values
nano .env.local # or use your preferred editor

# 5. Set up the database
# Create database in PostgreSQL
createdb automotive_store

# 6. Run database migrations and seed data
npm run db:setup # If available, or manually run SQL scripts

# 7. Start the development server
npm run dev

# 8. Open http://localhost:3000 in your browser
```

Detailed Setup Instructions

Step 1: Extract the Template Files

1. **Extract the downloaded ZIP file** to your desired location

```
bash
# Example for macOS/Linux
unzip e-commerce-automotive-store-temp-plate.zip
cd e-commerce-automotive-store-temp-plate
```

2. **Or clone from GitHub** (if you have repository access)

```
bash
git clone https://github.com/yourusername/e-commerce-automotive-store-temp-plate.git
cd e-commerce-automotive-store-temp-plate
```

Step 2: Install Dependencies

Install all required Node.js packages:

```
npm install
```

This will install:

- Next.js framework
- React and React DOM
- Stripe payment integration

- PostgreSQL database client
- NextAuth for authentication
- Email sending capabilities (Nodemailer)
- And all other dependencies

Expected output: You should see a progress bar and “added XXX packages” message.

Troubleshooting: If you encounter errors:

- Ensure Node.js version is 18+: `node --version`
- Clear npm cache: `npm cache clean --force`
- Delete `node_modules` and `package-lock.json`, then run `npm install` again

Step 3: Set Up Environment Variables

Environment variables configure your application for your specific setup.

1. Copy the example file:

```
bash
cp .env.local.example .env.local
```

2. Edit the `.env.local` file with your actual values:

```
bash
# Use your preferred text editor
nano .env.local
# or
code .env.local # If using VS Code
```

3. Fill in the required values (see [Environment Variables Explained](#) section below)

Step 4: Database Setup

4.1 Create PostgreSQL Database

Option A: Using Command Line

```
# Log into PostgreSQL
psql -U postgres

# Create the database
CREATE DATABASE automotive_store;

# Create a user (optional, for better security)
CREATE USER store_admin WITH PASSWORD 'your_secure_password';
GRANT ALL PRIVILEGES ON DATABASE automotive_store TO store_admin;

# Exit PostgreSQL
\q
```

Option B: Using pgAdmin

1. Open pgAdmin
2. Right-click on “Databases” → “Create” → “Database”
3. Name: `automotive_store`
4. Owner: `postgres` (or your custom user)
5. Click “Save”

4.2 Configure Database Connection

Update your `.env.local` file with the database connection string:

```
DATABASE_URL="postgresql://username:password@localhost:5432/automotive_store"
```

Replace:

- `username` - your PostgreSQL username (default: `postgres`)
- `password` - your PostgreSQL password
- `localhost` - your database host (use `localhost` for local development)
- `5432` - PostgreSQL port (default: 5432)
- `automotive_store` - your database name

Example:

```
DATABASE_URL="postgresql://postgres:mypassword123@localhost:5432/automotive_store"
```

4.3 Run Database Migrations

Set up the database schema:

```
# If migration scripts are provided
npm run db:migrate

# Or run the SQL schema file directly
psql -U postgres -d automotive_store -f scripts/schema.sql
```

4.4 Seed the Database (Optional)

Add sample data for testing:

```
# If seed script is provided
npm run db:seed

# Or run the seed SQL file directly
psql -U postgres -d automotive_store -f scripts/seed.sql
```

Step 5: Configure Stripe (Required for Payments)

For detailed Stripe setup instructions, see [STRIPE_SETUP_GUIDE.md](#) (`./STRIPE_SETUP_GUIDE.md`)

Quick setup:

1. Create a Stripe account at <https://stripe.com>
2. Get your API keys from the Stripe Dashboard
3. Add keys to `.env.local` :

```
env
```

```
STRIPE_SECRET_KEY=sk_test_...
```

```
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_...
```

```
STRIPE_WEBHOOK_SECRET=whsec_...
```

Step 6: Configure NextAuth Secret

Generate a secure secret for session encryption:

```
# Generate a random secret
openssl rand -base64 32
```

Copy the output and add it to `.env.local` :

```
NEXTAUTH_SECRET=your_generated_secret_here
```

Step 7: Configure Email Settings (Optional)

For order confirmation emails, configure your SMTP settings:

```
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=your.email@gmail.com
EMAIL_PASS=your_app_specific_password
EMAIL_FROM=noreply@yourstore.com
```

Gmail Setup:

1. Enable 2-factor authentication on your Google account
2. Generate an “App Password” in Google Account settings
3. Use the app password in `EMAIL_PASS`

Step 8: Start the Development Server

```
npm run dev
```

You should see:

```
✓ Ready in 3.2s
- Local:      http://localhost:3000
- Network:    http://192.168.1.X:3000
```

Step 9: Access Your Store

Open your browser and navigate to:

- **Storefront:** `http://localhost:3000`
- **Admin Panel:** `http://localhost:3000/admin` (login required)

Environment Variables Explained

Here’s a detailed explanation of each environment variable:

Authentication Variables

```
NEXTAUTH_SECRET=
```

Required: Yes

Description: Secret key for encrypting session tokens

How to generate: `openssl rand -base64 32`

Example: `NEXTAUTH_SECRET=abc123xyz789randomsecret`

```
NEXTAUTH_URL=http://localhost:3000
```

Required: Yes

Description: Base URL of your application

Development: `http://localhost:3000`

Production: Your actual domain (e.g., `https://yourstore.com`)

```
NEXT_PUBLIC_API_URL=http://localhost:3000
```

Required: Yes

Description: Base URL for API calls (client-side accessible)

Development: `http://localhost:3000`

Production: Your actual domain (e.g., `https://yourstore.com`)

Database Variables

```
DATABASE_URL="postgresql://user:password@localhost:5432/automotive_store"
```

Required: Yes

Description: PostgreSQL connection string

Format: `postgresql://USERNAME:PASSWORD@HOST:PORT/DATABASE_NAME`

Example: `postgresql://postgres:mypass@localhost:5432/automotive_store`

Stripe Payment Variables

```
STRIPE_SECRET_KEY=sk_test_...
```

Required: Yes (for payments)

Description: Stripe secret API key (server-side only)

Where to find: Stripe Dashboard → Developers → API Keys

Note: Use `sk_test_` for testing, `sk_live_` for production

```
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_...
```

Required: Yes (for payments)

Description: Stripe publishable key (client-side safe)

Where to find: Stripe Dashboard → Developers → API Keys

Note: Use `pk_test_` for testing, `pk_live_` for production

```
STRIPE_WEBHOOK_SECRET=whsec_...
```

Required: Yes (for payment webhooks)

Description: Webhook signing secret for verifying Stripe events

Where to find: Stripe Dashboard → Developers → Webhooks

Setup: See `STRIPE_SETUP_GUIDE.md` for detailed instructions

Cloudinary Variables (Optional)

```
CLOUDINARY_API_KEY=your_api_key  
CLOUDINARY_API_SECRET=your_api_secret
```

Required: No (only if using Cloudinary for image uploads)

Description: Cloudinary credentials for image management

Where to find: Cloudinary Dashboard → Settings → Access Keys

Email Configuration Variables

```
EMAIL_HOST=smtp.gmail.com  
EMAIL_PORT=587  
EMAIL_USER=your.email@gmail.com  
EMAIL_PASS=your_app_password  
EMAIL_FROM=noreply@yourstore.com
```

Required: No (only if sending order confirmation emails)

Description: SMTP server configuration for sending emails

Common providers:

- Gmail: `smtp.gmail.com:587`
- SendGrid: `smtp.sendgrid.net:587`
- Mailgun: `smtp.mailgun.org:587`

Testing Your Installation

1. Verify Homepage Loads

Navigate to `http://localhost:3000` and verify:

- ☒ Homepage displays correctly
- ☒ Product images load
- ☒ Navigation menu works
- ☒ No console errors in browser DevTools

2. Test Database Connection

Check the browser console for any database connection errors. You should see products loading if the database is connected.

3. Test Product Pages

- Click on a product
- Verify product details page loads
- Check that images and descriptions display correctly

4. Test Shopping Cart

- Add a product to cart
- Verify cart icon updates with item count
- Open cart and verify product is listed
- Test quantity increase/decrease

5. Test Stripe Integration (Test Mode)

Use Stripe test cards:

- Card Number: 4242 4242 4242 4242
- Expiry: Any future date (e.g., 12/34)
- CVC: Any 3 digits (e.g., 123)
- ZIP: Any 5 digits (e.g., 12345)

Proceed through checkout and verify:

- ☒ Checkout form loads
- ☒ Stripe payment form appears
- ☒ Test payment completes successfully
- ☒ Order confirmation page displays

6. Test Admin Panel (if applicable)

Navigate to `http://localhost:3000/admin` :

- ☒ Login page displays
- ☒ Can log in with admin credentials
- ☒ Admin dashboard loads
- ☒ Can view orders

Common Setup Issues

Issue 1: “Cannot connect to database”

Symptoms: Error messages about database connection failures

Solutions:

1. Verify PostgreSQL is running:

```
bash
# Check if PostgreSQL is running
psql -U postgres -c "SELECT version();"
```

1. Check your `DATABASE_URL` in `.env.local` :

- Correct username/password
- Correct database name
- Correct host and port

2. Ensure the database exists:

```
bash
psql -U postgres -l
```

3. Test connection manually:

```
bash
psql -U postgres -d automotive_store
```

Issue 2: “Module not found” errors

Symptoms: Import/require errors when starting the server

Solutions:

1. Delete `node_modules` and reinstall:


```
bash
rm -rf node_modules package-lock.json
npm install
```

1. Clear Next.js cache:

```
bash
rm -rf .next
npm run dev
```

Issue 3: Port 3000 already in use

Symptoms: Error: listen EADDRINUSE: address already in use :::3000

Solutions:

1. Kill the process using port 3000:

```
``bash
# Find the process
lsof -ti:3000

# Kill it
kill -9 $(lsof -ti:3000)
``
```

1. Or use a different port:

```
bash
PORT=3001 npm run dev
```

Issue 4: Stripe webhook not receiving events

Symptoms: Orders not completing after payment

Solutions:

1. Use Stripe CLI for local testing:

```
bash
stripe listen --forward-to localhost:3000/api/checkout/webhook
```

1. Copy the webhook signing secret to `.env.local`
2. See STRIPE_SETUP_GUIDE.md for detailed webhook setup

Issue 5: Images not loading

Symptoms: Broken image icons, 404 errors for images

Solutions:

1. Check `public/` folder contains images
2. Verify image paths in database
3. Check Next.js image configuration in `next.config.js`

Issue 6: Environment variables not loading

Symptoms: undefined values for environment variables

Solutions:

1. Ensure file is named `.env.local` (not `.env.local.txt`)
2. Restart the development server after changing `.env.local`
3. Verify variables start with `NEXT_PUBLIC_` if used in client-side code

Next Steps

Congratulations! 🎉 Your store is now set up. Here's what to do next:

1. Customize Your Store

→ See [CUSTOMIZATION_GUIDE.md](#) (./CUSTOMIZATION_GUIDE.md)

2. Configure Stripe for Production

→ See [STRIPE_SETUP_GUIDE.md](#) (./STRIPE_SETUP_GUIDE.md)

3. Deploy to Production

→ See [DEPLOYMENT_GUIDE.md](#) (./DEPLOYMENT_GUIDE.md)

4. Add Your Products

- Use the admin panel to add/edit products
- Import products via SQL scripts

5. Test Everything Thoroughly

- Test all user flows
- Test payment processing
- Test on different devices/browsers

Support

If you encounter issues not covered in this guide:

1. Check the [FAQ.md](#) (./FAQ.md) for common questions
2. Check the [troubleshooting.md](#) (./troubleshooting.md) guide
3. Review the [developer_guide.md](#) (./developer_guide.md) for technical details

Important Notes



Security:

- Never commit `.env.local` to version control
- Use strong passwords for production databases
- Keep your Stripe secret keys confidential
- Enable HTTPS in production



Testing:

- Always use Stripe test keys during development
- Test all features before going live
- Use test credit cards for payment testing



Performance:

- Optimize images before uploading
 - Use a CDN for production (e.g., Cloudinary, Vercel Edge Network)
 - Monitor database query performance
-

Last Updated: November 2024

Template Version: 1.0.0